

Human Crowd Detection and People Counting Using YOLOv8

1. Project Description

This project uses YOLOv8 and OpenCV to detect people in a video. The YOLOv8 pretrained model identifies objects belonging to the person class (class 0). For every video frame, the program detects people, draws a green bounding box around each detected person, displays the Person label, and calculates the total number of people present in that frame. The current people count is displayed at the top of the video.

2. Technologies Used

- Python
- OpenCV (cv2)
- Ultralytics YOLOv8
- Computer Vision
- Object Detection

3. Step-by-Step Code Explanation

Step 1 — Import Libraries

```
import cv2
from ultralytics import YOLO
```

OpenCV is used for video processing, drawing, displaying frames, and keyboard input. YOLO is imported from Ultralytics for object detection.

Step 2 — Load YOLOv8 Model

```
model = YOLO("yolov8s.pt")
```

Loads the pretrained YOLOv8 Small model.

Step 3 — Open the Video

```
cap = cv2.VideoCapture("croud.mp4")
```

Opens the input video file so that it can be processed frame by frame.

Step 4 — Start Processing Loop

```
while True:
```

Creates a continuous loop that processes video frames until the video ends or ESC is pressed.

Step 5 — Read a Frame

```
ret, frame = cap.read()
```

Reads the next video frame. 'ret' indicates whether the frame was successfully read, while 'frame' contains the image.

Step 6 — Check for End of Video

```
if not ret:  
    break
```

Stops the loop when another frame cannot be read.

Step 7 — Detect Only People

```
results = model(frame, classes=[0])
```

Runs YOLO detection on the current frame. Class 0 is used to detect people only.

Step 8 — Initialize People Counter

```
people_count = 0
```

Starts the people counter at zero for each current frame.

Step 9 — Process YOLO Results

```
for result in results:  
    boxes = result.boxes
```

Gets the bounding-box information returned by YOLO.

Step 10 — Process Each Detected Person

```
for box in boxes:
```

Processes every detected person individually.

Step 11 — Get Bounding-Box Coordinates

```
x1, y1, x2, y2 = map(int, box.xyxy[0])
```

Gets the top-left and bottom-right coordinates of each detected person's bounding box.

Step 12 — Draw Bounding Box

```
cv2.rectangle(frame, (x1, y1), (x2, y2), (0,255,0), 2)
```

Draws a green rectangle around the detected person. The final value 2 is the line thickness.

Step 13 — Add Person Label

```
cv2.putText(frame, "Person",
            (x1, y1-10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6,
            (0,255,0), 2)
```

Displays the "Person" label above the bounding box.

Step 14 — Increase the Count

```
people_count += 1
```

Increases the people count by one for every detected person.

Step 15 — Display People Count

```
cv2.putText(frame,
            f"People Count: {people_count}",
            (20,40),
            cv2.FONT_HERSHEY_SIMPLEX, 1,
            (0,0,255), 2)
```

Displays the number of detected people at the top of the video.

Step 16 — Display the Video

```
cv2.imshow("People Detection", frame)
```

Displays the processed frame in an OpenCV window named People Detection.

Step 17 — Stop with ESC

```
if cv2.waitKey(1) == 27:
    break
```

Checks for keyboard input. ASCII code 27 represents the ESC key, which stops the program.

Step 18 — Release Resources

```
cap.release()
cv2.destroyAllWindows()
```

Releases the video resource and closes all OpenCV windows.

4. Complete Program

```
import cv2
from ultralytics import YOLO

model = YOLO("yolov8s.pt")

cap = cv2.VideoCapture("croud.mp4")

while True:
    ret, frame = cap.read()

    if not ret:
        break

    results = model(frame, classes=[0])

    people_count = 0

    for result in results:
        boxes = result.boxes

        for box in boxes:
            x1, y1, x2, y2 = map(int, box.xyxy[0])

            cv2.rectangle(frame, (x1, y1), (x2, y2), (0,255,0), 2)

            cv2.putText(frame, "Person",
                        (x1, y1-10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                        (0,255,0), 2)

            people_count += 1

    cv2.putText(frame,
                f"People Count: {people_count}",
                (20,40),
                cv2.FONT_HERSHEY_SIMPLEX, 1,
                (0,0,255), 2)

    cv2.imshow("People Detection", frame)

    if cv2.waitKey(1) == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

5. Project Flow

START → Import OpenCV + YOLO → Load YOLOv8 Model → Open Video → Read Frame → Check Frame → Detect Person (Class 0) → Draw Bounding Box → Add Person Label → Count People → Display Count → Show Frame → Check ESC → Repeat / END

7. Viva Answer

My project is a Human Crowd Detection and People Counting system. It uses the YOLOv8 pretrained object-detection model and OpenCV. The input is a video processed frame by frame. YOLOv8 detects only the person class, and the program draws bounding boxes around detected people. It counts the detected people in the current frame and displays the count on the video. The program can be stopped by pressing the ESC key.

8. Important Note

This code performs per-frame detection and counting, not persistent multi-object tracking. Therefore, the displayed count represents the number of people detected in the current frame, not unique people across the entire video.